

Unit-5 - Formal and Symbolic Programming Paradigm**12 Hour**

Automata Based programming Paradigm: Finite Automata – DFA and NFA; Implementing using Automaton Library - Symbolic Programming Paradigm: Algebraic manipulations and calculus; Sympy Library - Event Programming Paradigm: Event Handler, Trigger functions and Events – Tkinter Library. Develop a python-based project dissertation based on the programming paradigm.

Learning Resources	1. Elad Shalom, A Review of Programming Paradigms throughout the History: With a suggestion Toward a Future Approach, Kindle Edition, 2018 2. Maurizio Gabrielli, Simone Martini, Programming Languages: Principles and Paradigms, 2010.	3. Herbert Schildt, Java: The Complete Reference Seventh Edition, 2016. 4. Mark Lutz, Programming Python: Powerful Object-Oriented Programming, 2011.
---------------------------	--	---

Learning Assessment

Learning Assessment		Bloom's Level of Thinking	Continuous Learning Assessment (CLA)						Final Examination (0% weightage)	
			Formative CLA-1 Average of unit test (20%)		Project Based Learning CLA-2 (60%)		Report and Viva Voce (20% weightage)			
			Theory	Practice	Theory	Practice	Theory	Practice		
Level 1		Remember	30%	-	--	20%	-	10%	-	-
Level 2		Understand	30%	-	-	20%	-	10%	-	-
Level 3		Apply	20%	-	-	20%	-	10%	-	-
Level 4		Analyze	20%	-	-	20%	-	10%	-	-
Level 5		Evaluate	-	-	-	10%	-	30%	-	-
Level 6		Create	-	-	-	10%	-	30%	-	-
		Total	100 %		100 %		100 %		-	

Course Designers

Experts from Industry		Experts from Higher Technical Institutions	Internal Experts
1. Mr. N. Venkatesh, Tech Lead, Honeywell, Bengaluru, Karnataka, India		1. Dr. Sudeepta Mishra, Assistant Professor, Computer Science and Engineering, Indian Institute of Information Technology, Ropar, Punjab.	1. Dr Ramkumar J, SRMIST

Course Code	21CSC204J	Course Name	DESIGN AND ANALYSIS OF ALGORITHMS		Course Category	C	PROFESSIONAL CORE				L	T	P	C
											3	0	2	4

Pre-requisite Courses	Nil	Co- requisite Courses	Nil	Progressive Courses	Nil
Course Offering Department	School of Computing	Data Book / Codes / Standards		Nil	

Course Learning Rationale (CLR):		The purpose of learning this course is to:											
CLR-1:	design efficient algorithms in solving complex real time problems												
CLR-2:	analyze various algorithm design techniques to solve real time problems in polynomial time												
CLR-3:	utilize various approaches to solve greedy and dynamic algorithms												
CLR-4:	utilize back tracking and branch and bound paradigms to solve exponential time problems												
CLR-5:	analyze the need of approximation and randomization algorithms, utilize the importance Non polynomial algorithms												
Course Outcomes (CO):		At the end of this course, learners will be able to:											
CO-1:	apply efficient algorithms to reduce space and time complexity of both recurrent and non-recurrent relations												
CO-2:	solve problems using divide and conquer approaches												
CO-3:	apply greedy and dynamic programming type's techniques to solve polynomial time problems												
CO-4:	create exponential problems using backtracking and branch and bound approaches												
CO-5:	interpret various approximation algorithms and interpret solutions to evaluate P type, NP Type, NPC, NP Hard problems												

Program Outcomes (PO)												Program Specific Outcomes		
1	2	3	4	5	6	7	8	9	10	11	12			
Engineering Knowledge	Problem Analysis	Design/development of solutions	Conduct investigations of complex problems	Modern Tool Usage	The engineer and society	Environment & Sustainability	Ethics	Individual & Team Work	Communication	Project Mgt. & Finance	Life Long Learning	PSO-1	PSO-2	PSO-3
2	1	2	1	-	-	-	-	-	3	-	3	3	1	-
2	1	2	1	-	-	-	-	-	3	-	3	3	1	-
2	1	2	1	-	-	-	-	-	3	-	3	3	1	-
2	1	2	1	-	-	-	-	-	3	-	3	3	1	-
2	1	2	1	-	-	-	-	-	3	-	3	3	1	-

Unit-1 - Introduction to Algorithm Design		15 Hour										15 Hour	
Fundamentals of Algorithms- Correctness of algorithm - Time complexity analysis - Insertion sort-Line count, Operation count Algorithm Design paradigms - Designing an algorithm And its analysis-Best, Worst and Average case - Asymptotic notations Based on growth functions. $O, O(\theta), \omega, \Omega$ - Mathematical analysis - Induction, Recurrence relations - Solution of recurrence relations - Substitution method - Solution of recurrence relations - Recursion tree - Solution of recurrence relations - examples.													
Unit-2 - Divide and Conquer		15 Hour										15 Hour	
Maximum Subarray Problem Binary Search - Complexity of binary search Merge sort - Time complexity analysis -Quick sort and its Time complexity analysis Best case, Worst case, Average case analysis - Strassen's Matrix multiplication and its recurrence relation - Time complexity analysis of Merge sort - Largest sub-array sum - Time complexity analysis of Largest sub- array sum - Master Theorem Proof - Master theorem examples - Finding Maximum and Minimum in an array - Time complexity analysis-Examples - Algorithm for finding closest pair problem - Convex Hull problem													
Unit-3 - Greedy and Dynamic Programming		15 Hour										15 Hour	
- Examples of problems that can be solved by using greedy and dynamic approach Huffman coding using greedy approach Comparison of brute force and Huffman method of encoding - Knapsack problem using greedy approach Complexity derivation of knapsack using greedy - Tree traversals - Minimum spanning tree – greedy Kruskal's algorithm - greedy - Minimum spanning tree - Prim's algorithm Introduction to dynamic programming - 0/1 knapsack problem - Complexity calculation of knapsack problem - Matrix chain multiplication using dynamic programming - Complexity of matrix chain multiplication - Longest common subsequence using dynamic programming - Explanation of LCS with an example - Optimal binary search tree (OBST) using dynamic programming - Explanation of OBST with an example.													